<!-- == --> <!-- CONTINUATION --> <!-- Picks up immediately after: --> <!-- "In fact, it is important to note that processes are --> <!-- completely unaware of the fact they are sharing the CPU --> <!-- with other processes." --> <!-- == -->

But so far we've been looking at these pieces in isolation -- the memory layout in `mm_struct`, the process states in `__state`, the saved registers in `pt_regs`. In reality, the kernel needs to bundle *all* of this together into a single data structure for each process. And it does.

The Process Control Block

In OS terminology, the data structure that holds everything the kernel needs to know about a process is called the **Process Control Block (PCB)**. In Linux, the PCB is a struct called `task_struct`, and it is enormous. As of the latest kernel, it spans well over 700 lines of fields. It contains (or points to) essentially everything we just covered, and much more.

Here are some of the key fields (heavily simplified from the actual source):

```c
```

```
struct task_struct {
    /* ----- process state ----- */
    unsigned int              __state;       /* TASK_RUNNING, TASK_INTERRUPTIBLE, et

    /* ----- identity ----- */
    pid_t                     pid;           /* process ID */
    pid_t                     tgid;          /* thread group ID (we'll come back to

    /* ----- memory ----- */
    struct mm_struct          *mm;           /* pointer to the address space layout

    /* ----- CPU state (saved/restored on context switch) ----- */
    struct thread_struct      thread;        /* architecture-specific register state

    /* ----- scheduling ----- */
    int                       prio;          /* dynamic priority */
    int                       static_prio;   /* base priority set by nice value */
    const struct sched_class  *sched_class;  /* scheduling policy (CFS, RT, etc.) */

    /* ----- file system ----- */
    struct files_struct       *files;        /* table of open file descriptors */

    /* ----- parent/child relationships ----- */
    struct task_struct        *parent;       /* pointer to parent process */
    struct list_head          children;      /* list of child processes */

    /* ----- signals ----- */
    struct signal_struct      *signal;       /* signal handling info */

    /* ----- and hundreds more fields... ----- */
};
```

This is where everything comes together. Remember our questions from the Multiprogramming section?

- *How do we stop one program and resume it later without losing its state?* The kernel saves the process's register state into the `thread` field of its `task_struct`, and restores it when the process is scheduled again.
- *How do we know where to resume execution?* The saved register state includes the instruction pointer (`ip`), which tells the CPU exactly which instruction to execute next.

- *How do we prevent one program from interfering with another program's memory?* Each process has its own `mm_struct` (pointed to by the `mm` field), which defines its own virtual address space. The kernel configures the hardware's memory management unit (MMU) so that one process literally cannot see or touch another process's memory.

The `task_struct` is, in many ways, the most important data structure in the entire kernel. The kernel maintains a list of all `task_struct` instances -- one per process -- and uses this list to make scheduling decisions, deliver signals, manage memory, and everything else that involves processes.

I encourage you to browse the full `task_struct` definition on Bootlin (a much friendlier way to browse kernel source than raw GitHub). You won't understand every field, but you'll be surprised how many of them make sense now.

The Process API

Now that we know what a process *is* at the kernel level, let's look at how processes are created and managed. In UNIX-like systems, process management is done through a small set of system calls. The three most important are `fork()`, `exec()`, and `wait()`.

`fork()` creates a new process by *duplicating* the calling process. The new process (called the **child**) is an almost-exact copy of the original (called the **parent**) -- it gets its own `task_struct`, its own copy of the address space, its own file descriptor table, but all the *values* start out the same. The key difference: `fork()` returns the child's PID to the parent, and `0` to the child. This is how each process knows which one it is after the fork.

`exec()` (technically a family of functions: `execl`, `execvp`, `execve`, etc.) replaces the current process's address space with a brand-new program. The text segment is overwritten with the new program's instructions, the heap and stack are re-initialized, but the process keeps the same PID and the same `task_struct`. It's the same process wearing a different program's clothes.

`wait()` allows a parent process to wait for a child to finish executing. When a child process terminates, its `task_struct` isn't immediately destroyed -- it lingers in the **zombie** state (remember `EXIT_ZOMBIE` from the kernel source above?) until the parent calls `wait()` to collect the child's exit status. If the parent never calls `wait()`, the zombie sticks around, which is why long-lived server processes need to handle child cleanup carefully.

This pattern -- `fork()` followed by `exec()` followed by `wait()` -- is the foundation of how UNIX shells work. When you type `ls` in your terminal, the shell:

1. Calls `fork()` to create a child process.
2. The child calls `exec()` to replace itself with the `ls` program.
3. The parent (the shell) calls `wait()` to block until `ls` finishes.
4. When `ls` exits, `wait()` returns, and the shell is ready for your next command.

<CodeFigure caption="The classic fork/exec/wait pattern that UNIX shells use to run programs."> ```c #include <stdio.h> #include <stdlib.h> #include <unistd.h> #include <sys/wait.h>

int main(void)
{
printf("parent process (pid: %d)\n", getpid());

```c
pid_t pid = fork();  /* create a child process */

if (pid < 0) {
    /* fork failed */
    fprintf(stderr, "fork failed\n");
    return 1;

} else if (pid == 0) {
    /* we are the child process -- fork() returned 0 */
    printf("child process (pid: %d)\n", getpid());

    /* replace this process with the "ls" program */
    char *args[] = { "ls", "-la", NULL };
    execvp(args[0], args);

    /* if we get here, exec failed */
    fprintf(stderr, "exec failed\n");
    return 1;

} else {
    /* we are the parent process -- fork() returned the child's PID */
    printf("parent waiting for child (pid: %d)...\n", pid);

    int status;
    waitpid(pid, &status, 0);  /* block until child exits */

    printf("child exited with status %d\n", WEXITSTATUS(status));
}

return 0;
```

}

</CodeFigure>

One subtlety worth noting: `fork()` does not actually copy the entire address space immediately.
Modern kernels use a technique called **copy-on-write (COW)** -- the parent and child initially
share the same physical memory pages, marked as read-only. Only when one of them tries to *write*
to a page does the kernel create a separate copy for the writer. This makes `fork()` much cheaper
than a full memory copy, which matters because many `fork()` calls are immediately followed by `exec()`,
which throws away the address space anyway.

Limited Direct Execution

We've now covered what a process is, what its state looks like, and how processes are created.
But there's a fundamental question we've been skating past: **how does the kernel actually
get control of the CPU?**

Think about it. The kernel is just software -- it needs the CPU to run, just like any other program.
But if a user process is running on the CPU, executing its instructions at full speed, the kernel
isn't running. So how does the OS ever get a chance to make scheduling decisions, save register
state, or do anything at all?

This is the central tension of what OSTEP calls **Limited Direct Execution**. The "direct execution"
part means we want programs to run directly on the CPU hardware with no overhead -- no interpreter,
no emulation, just raw native execution for maximum speed. The "limited" part means we need the
OS to retain control so that processes can't do whatever they want.

Solving this requires cooperation between the OS and the hardware.

User Mode and Kernel Mode

Modern processors support (at minimum) two **privilege levels** or **modes** of

execution:

- **User mode** (also called Ring 3 on x86): the restricted mode where normal programs run. In user
  mode, a process can execute its own instructions and access its own memory, but it *cannot*
  directly access hardware, modify page tables, disable interrupts, or do anything that could
  affect other processes or the system as a whole. If it tries, the CPU raises an exception.

- **Kernel mode** (also called Ring 0 on x86): the privileged mode where the OS kernel runs. In
  kernel mode, code has unrestricted access to the entire machine -- all memory, all hardware,
  all CPU instructions.

(x86 technically has four rings, 0 through 3, but virtually all modern operating systems only
use Ring 0 and Ring 3. If you want to go down that rabbit hole, the
[Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A, Section 5](https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html)
covers the full protection model. For a more approachable treatment, Gustavo Duarte's
["CPU Rings, Privilege, and Protection"](https://manybutfinite.com/post/cpu-rings-privilege-and-protection/)
is excellent.)

This two-mode split is the foundation of process isolation. Every instruction your process executes
runs in user mode. It cannot escape user mode on its own. The only way to transition into kernel
mode is through a small number of controlled entry points.

Traps and System Calls

So how does a user-mode process ask the kernel for anything? Remember all those system calls
we've been using -- `open()`, `read()`, `fork()`, `malloc()` (which calls `mmap()` or `sbrk()`
underneath)? Each one of these needs to execute kernel code, which means each one needs to
transition from user mode to kernel mode.

The mechanism is called a **trap** (also sometimes called a software interrupt or a syscall
instruction, depending on the architecture). Here's the flow:

1. The user process executes a special instruction that triggers a trap. On x86-64, this
   is the `syscall` instruction (older systems used `int 0x80`).
2. The hardware *immediately* does three things:
   - Switches the CPU from user mode to kernel mode (privilege escalation).
   - Saves the current instruction pointer and a few other registers so the kernel
     knows where to return.
   - Jumps to a predetermined address in kernel code.
3. The kernel's **trap handler** examines which system call was requested (the syscall
   number is passed in a register, typically `rax` on x86-64), validates the arguments,
   performs the requested operation, and places the return value back in a register.
4. The kernel executes a **return-from-trap** instruction (`sysret` on x86-64),
   which restores the saved state, drops the CPU back to user mode, and resumes the
   user process right after the original `syscall` instruction.

The critical security property here is that the user process *cannot choose* what kernel code
runs. The hardware jumps to an address specified in a **trap table** (also called the Interrupt
Descriptor Table, or IDT, on x86). The kernel sets up this table at boot time, before any user
process ever runs. So even though the user process *triggers* the transition to kernel mode,
it can only enter the kernel at carefully controlled entry points.

This is why, for example, a user process can call `read()` to read from a file descriptor it owns,
but it can't directly read from another process's memory or write to arbitrary disk sectors. The
kernel validates every request that comes through the trap handler.

The Problem of Cooperative Scheduling

System calls give the kernel control whenever a process *voluntarily* asks for

```
something.  But
what happens if a process never makes a system call?  What if it just sits in a
tight loop:
```c
while (1) {
    /* compute something forever, never calling the OS */
    x = x + 1;
}
```

In the earliest multiprogramming systems, this was a real problem. The OS relied on processes to *cooperate* by periodically yielding control -- either through system calls or through an explicit `yield()` call. This is called **cooperative scheduling** (or **non-preemptive scheduling**), and it works exactly as well as you'd expect a system based on trust to work. A single buggy or malicious program could monopolize the CPU forever.

Timer Interrupts

The solution is hardware-assisted **preemption**. Modern processors have a **timer device** that can be programmed to fire an **interrupt** at a regular interval (typically every few milliseconds). When the timer fires:

1. The hardware *forcibly* pauses whatever is currently running -- it doesn't matter if the process is in the middle of a computation, in the middle of an instruction sequence, it doesn't matter. The timer interrupt takes priority.
2. The CPU saves the current state (similar to a trap) and switches to kernel mode.
3. The kernel's **timer interrupt handler** runs. This is where the scheduler gets its chance to decide: should we keep running the current process, or switch to a different one?
4. If the kernel decides to switch, it performs a **context switch** (which we'll cover in the next section). If not, it returns from the interrupt and the process continues, completely unaware that it was briefly interrupted.

This is called **preemptive scheduling**, and it's what every modern OS uses. The key insight is that the kernel doesn't need to *trust* processes to yield the CPU -- the hardware *forces* the issue at regular intervals, giving the kernel opportunities to make scheduling decisions regardless of what the running process is doing.

Between traps (voluntary, triggered by system calls) and timer interrupts (involuntary, triggered by hardware), the kernel has two reliable mechanisms for regaining control of the CPU.
These are the only two ways control transitions from a user process back to the kernel.

Context Switching
```

We've now established *when* the kernel gets control (traps and timer interrupts) and *why* it needs control (to manage processes and maintain the illusion of simultaneous execution). Now let's look at *what it does* with that control when it decides to switch from one process to another.

The operation is called a **context switch**, and at its core it's mechanical: save the state of the currently-running process, and restore the state of the next process to run. But let's be precise about what "state" means here, because there are actually *two* levels of saving and restoring that happen.

When a timer interrupt fires (or a trap occurs), the *hardware* automatically saves a small set of registers -- the instruction pointer, the stack pointer, the flags register, and a few segment registers -- onto the process's **kernel stack**. (Every process has its own kernel stack, separate from the user-mode stack you're familiar with. This is where the kernel does its work on behalf of the process.)

If the kernel decides to continue running the same process, it simply returns from the interrupt, the hardware restores those registers, and the process continues. No context switch.

But if the kernel decides to switch to a different process, the *software* (the kernel itself) performs the second level of saving: it takes *all* the remaining register state of the current process and stores it in that process's `task_struct`. It then loads the saved register state from the *next* process's `task_struct`, switches to the next process's kernel stack, and executes a return-from-trap. The hardware restores the next process's registers, drops back to user mode, and the next process picks up right where it left off.

We can look at the actual Linux implementation. In `kernel/sched/core.c`, the `context_switch()` function handles the high-level logic:

```c
```

```
/*
 * context_switch - switch to the new MM and the new thread's register state.
 *
 * (heavily simplified from the actual source)
 */
static __always_inline struct rq *
context_switch(struct rq *rq, struct task_struct *prev,
               struct task_struct *next, ...)
{
    /* ... */

    /* switch the virtual memory mapping to the next process's address space */
    if (!next->mm) {
        /* kernel thread -- borrow the previous task's mm */
        next->active_mm = prev->active_mm;
    } else {
        /* user process -- switch to its address space */
        switch_mm_irqs_off(prev->active_mm, next->mm, next);
    }

    /* ... */

    /* switch the register state: save prev's registers, restore next's */
    switch_to(prev, next, prev);

    return finish_task_switch(prev);
}
```

And the architecture-specific `switch_to` macro (on x86-64) ultimately calls `__switch_to_asm`, which is written in assembly because it needs to manipulate registers directly:

```asm
```

```
/*
 * %rdi: prev task (the process we're switching away from)
 * %rsi: next task (the process we're switching to)
 *
 * (simplified from arch/x86/entry/entry_64.S)
 */
SYM_FUNC_START(__switch_to_asm)
    /* save callee-saved registers of the prev task onto its kernel stack */
    pushq   %rbp
    pushq   %rbx
    pushq   %r12
    pushq   %r13
    pushq   %r14
    pushq   %r15

    /* switch the kernel stack pointer from prev's stack to next's stack */
    movq    %rsp, TASK_threadsp(%rdi)    /* save prev's stack pointer */
    movq    TASK_threadsp(%rsi), %rsp    /* load next's stack pointer */

    /* restore callee-saved registers of the next task from its kernel stack */
    popq    %r15
    popq    %r14
    popq    %r13
    popq    %r12
    popq    %rbx
    popq    %rbp

    jmp     __switch_to                  /* finish up in C code */
SYM_FUNC_END(__switch_to_asm)
```

Read that assembly carefully -- it's remarkably elegant for what it accomplishes. The trick is the stack pointer swap in the middle. We push the current process's registers onto its stack, swap the stack pointer to the next process's stack, and then *pop* -- but now we're popping from a *different stack*, so we get the *next* process's previously-saved registers. When this function returns, it returns into the next process's execution flow. The stack swap *is* the context switch.

Context Switches Are Not Free

It's easy to treat context switches as "the kernel does some bookkeeping and moves on," but they have real, measurable costs beyond just the time it takes to save and restore registers:

TLB flushes: When the kernel switches to a new process's address space (via `switch_mm`), the processor's Translation Lookaside Buffer (TLB) -- a cache of virtual-to-physical address translations -- may need to be partially or fully flushed. The new process will experience TLB

misses for a while as it rebuilds its working set of translations, and each miss means a page table walk in hardware.

Cache pollution: The new process's code and data will start evicting the previous process's cache lines from the CPU's L1/L2/L3 caches. Until the new process "warms up" its cache, memory accesses will be slower.

Pipeline stalls: Modern CPUs have deep instruction pipelines with branch predictors, prefetchers, and other speculative execution machinery. A context switch resets all of this, and the new process starts with a "cold" pipeline until the predictor learns its patterns.

If you want to measure context switch cost on your own machine, the `lmbench` benchmark suite (McVoy & Staelin, USENIX 1996) has a `lat_ctx` test that does exactly this using two processes communicating over pipes. Typical modern numbers are in the low single-digit microseconds for the switch itself, but the indirect costs (cache/TLB misses) can dominate for memory-intensive workloads.

These costs are worth keeping in mind, because they are one of the key motivations for seeking lighter-weight alternatives to full process context switches. Which brings us to threads.

Threads

Let's zoom back out for a second. We now have a complete picture of how the OS virtualizes the CPU:

- Processes provide the abstraction (each one thinks it has the whole machine).
- Limited Direct Execution provides the control mechanism (user mode restricts what processes can do; traps and timer interrupts give the kernel opportunities to intervene).
- Context switching provides the mechanics (save one process's state, restore another's).

This works. But there's a problem. Imagine you're building a web server. For every incoming connection, you could `fork()` a new process to handle it. Each process gets its own address space, its own file descriptor table, its own everything. This is safe and isolated, but it's *heavy*. Every `fork()` allocates a new `task_struct`, sets up copy-on-write page mappings, and later, every context switch between these processes may require a TLB flush and cache warmup.

What if many of those processes are doing similar work, operating on shared data structures, and don't actually *need* full isolation from each other? What if they could share the same address space -- the same heap, the same global variables, the same code -- but still execute independently?

This is the idea behind **threads**. A thread is a unit of execution *within* a process. Multiple threads in the same process share:

- The **text** segment (same code)

- The **data** segment (same global variables)
- The **heap** (same dynamically allocated memory)
- Open **file descriptors**
- The process's **PID** (from the outside, they look like one process)

But each thread has its own:

- **Stack** (each thread needs its own call stack for function-local variables and return addresses)
- **Registers** (including its own instruction pointer -- each thread is at a different point in execution)
- **Thread ID** (TID)

<Figure src="/media/multi_threaded_address_space.png" alt="Diagram of a multi-threaded address space showing shared text, data, and heap segments, but separate stacks for each thread." caption="multi-threaded address space: threads share everything except their stacks" />

In Linux, threads are implemented in a way that might surprise you: a thread is just a `task_struct` that happens to share its `mm_struct` (and other resources) with another `task_struct`. The kernel doesn't have a fundamentally different data structure for threads -- it uses the same `task_struct` for both. This is why the `task_struct` has both a `pid` field (which is actually the thread ID) and a `tgid` field (thread group ID, which is the PID of the main thread -- what userspace sees as the "process ID").

Threads are created in Linux using the `clone()` system call (which `fork()` is also built on top of), with flags that specify which resources to share:

```c
```

```
/*
 * Flags passed to clone() that determine what is shared between
 * parent and child.  (from include/uapi/linux/sched.h)
 *
 * When creating threads, the threading library (e.g., pthreads) passes
 * most of these flags so the new task shares the parent's resources.
 * When creating processes (fork), none of these are passed, so the child
 * gets its own copies.
 */
#define CLONE_VM      0x00000100  /* share memory space (address space) */
#define CLONE_FS      0x00000200  /* share filesystem info */
#define CLONE_FILES   0x00000400  /* share open file descriptors */
#define CLONE_SIGHAND 0x00000800  /* share signal handlers */
#define CLONE_THREAD  0x00010000  /* same thread group (share PID) */
/* ... and many more */
```

When you call `fork()`, the kernel calls `clone()` with *no* sharing flags -- the child gets its own copy of everything. When a threading library like pthreads creates a new thread, it calls `clone()` with `CLONE_VM | CLONE_FILES | CLONE_FS | CLONE_SIGHAND | CLONE_THREAD` (and others) -- the new task shares almost everything with its parent.

This brings us back to why context switches between threads are cheaper than between processes. When the kernel switches between two threads in the *same* process, the `switch_mm` step in `context_switch()` is essentially a no-op -- both threads share the same `mm_struct`, so there's no address space to switch, no TLB flush required. The kernel still needs to save and restore registers (the `switch_to` assembly code is identical), but skipping the memory mapping switch eliminates a significant portion of the cost.

But -- and this is the seed for the rest of the series -- thread context switches are still **not free**. We still save and restore registers. We still disrupt the pipeline. We still pollute the cache. And more importantly, threads introduce a whole new class of problems: because they share memory, concurrent access to shared data requires careful **synchronization** (locks, mutexes, condition variables) to avoid data races and corruption. We will cover these problems in a later part of the series.

For now, the important takeaway is this: threads are the kernel's answer to "processes are too expensive for workloads that need many concurrent tasks operating on shared data." They're lighter-weight, but they're not weightless. And the costs they carry -- even the reduced costs -- become the motivation for the even-lighter-weight concurrency mechanisms we'll eventually arrive at: event loops, non-blocking I/O, and `async`/`await`.

Wrapping Up

Let's step back and look at where we started and where we've arrived.

We began with a single CPU executing one instruction at a time, and a program that had to wait -- doing nothing -- while I/O completed. The waste was intolerable, so we invented **multiprogramming**: load multiple programs into memory and switch between them when one blocks.

That simple idea cascaded into an enormous amount of complexity. We needed an abstraction to represent a running program (the **process**, backed by `task_struct`), a way to track its full execution state (the **Process Control Block**), a mechanism for the OS to safely regain control (user/kernel mode, **traps**, and **timer interrupts**), a procedure for switching between processes (**context switching**), and a lighter-weight variant for when full process isolation is overkill (**threads**).

All of this machinery exists to create one illusion: that many things are happening at once on a machine that can really only do one thing at a time (per core). And at the heart of it all is the **Blocked state** -- a process sitting idle, waiting for I/O, unable to make progress. Everything we've built is a response to that waste.

But here's the question we haven't answered yet: when there are many processes (or threads) ready to run, **who goes next?** How does the kernel decide which one gets the CPU? And what happens when a process initiates I/O -- how exactly does the OS find out when the I/O completes, and what are the different strategies for handling it?

These questions -- **scheduling** and **I/O** -- are where we're headed in Part 2. The scheduling problem will show us the tradeoffs between responsiveness and throughput, and the I/O question will begin to reveal why simply blocking a thread and waiting isn't always the best answer. That's where the road to `async` really begins.

Further Reading

- Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* -- Chapters 4 (Processes), 5 (Process API), 6 (Limited Direct Execution), and 26 (Concurrency: An Introduction) cover everything in this post in more depth.
- Gustavo Duarte, "CPU Rings, Privilege, and Protection" -- An accessible deep dive into x86 privilege levels.
- The Linux kernel source on Bootlin -- Browse `task_struct`, `mm_struct`, and the scheduler source with cross-references and syntax highlighting.
- The xv6 operating system and its companion book -- A minimal, readable teaching OS that implements everything covered in this post.

- McVoy & Staelin, "lmbench: Portable Tools for Performance Analysis" (USENIX 1996) -- The benchmark suite for measuring context switch latency and other OS primitives.